

Name of Experiment: A C Program to Convert an Expression from Infix to Postfix.

Introduction: The program converts an **infix expression** into a **postfix expression** using a **stack**. It follows operator precedence and parentheses rules to generate the correct postfix form.

Example: Infix : A+B*C
 Postfix: ABC*+

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

typedef struct {
    int top;
    char items[MAX];
} Stack;

void initStack(Stack* s) {
    s->top = -1;
}

int isEmpty(Stack* s) {
    return s->top == -1;
}

int isFull(Stack* s) {
    return s->top == MAX - 1;
}

void push(Stack* s, char c) {
    if (isFull(s)) {
        printf("Stack overflow\n");
        return;
    }
    s->items[++(s->top)] = c;
}

char pop(Stack* s) {
    if (isEmpty(s)) {
        return '\0';
    }
    return s->items[(s->top)--];
}

char peek(Stack* s) {
    if (isEmpty(s)) {
        return '\0';
    }
    return s->items[s->top];
}

int precedence(char c) {
    if (c == '^')
        return 3;
    else if (c == '*' || c == '/')
        return 2;
    else if (c == '+' || c == '-')
        return 1;
    else
        return -1;
}
```

```

void infixToPostfix(char* infix, char* postfix) {
    stack s;
    initStack(&s);

    int i, j = 0;
    char c;

    for (i = 0; infix[i] != '\0'; i++) {
        c = infix[i];

        if (isalnum(c)) {
            postfix[j++] = c;
        }
        else if (c == '(') {
            push(&s, c);
        }
        else if (c == ')') {
            while (!isEmpty(&s) && peek(&s) != '(') {
                postfix[j++] = pop(&s);
            }
            pop(&s); // Remove '('
        }
        else {
            while (!isEmpty(&s) &&
                ((precedence(c) < precedence(peek(&s))) ||
                 (precedence(c) == precedence(peek(&s)) && c !=
                  '^')))) {
                postfix[j++] = pop(&s);
            }
            push(&s, c);
        }
    }

    while (!isEmpty(&s)) {
        postfix[j++] = pop(&s);
    }

    postfix[j] = '\0';
}

int main() {
    char infix[MAX], postfix[MAX];

    printf("Enter infix expression: ");

    if (scanf("%s", infix) != 1) {
        return 1;
    }

    infixToPostfix(infix, postfix);

    printf("Postfix expression: %s\n", postfix);

    return 0;
}

```

Output:

```

Enter infix expression: 12+2+4-2
Postfix expression: 122+4+2-

```

Discussion: The program successfully converted the infix expression:

$$12+2+4-2$$

into the postfix expression:

$$122+4+2-$$

In the postfix form, operands are written before operators.

The program scanned the expression from left to right and used a stack to maintain operator precedence and order of execution.

Here:

- Numbers were added directly to the postfix expression.
- Operators (+ and -) were pushed and popped according to precedence rules.
- Since + and - have the same precedence, operations were processed sequentially from left to right.

The output confirms that the infix-to-postfix conversion algorithm worked correctly using stack operations.